Program 1

Problem Statement
Combine several work/energy calculation classes (force-displacement, power-time, angle-force, kinetic, gravitational, elastic, thermodynamic, rotational) into a single class using multiple inheritance, compute each value, and report which is the greatest.

Source Code:

```
#include <bits/stdc++.h>
using namespace std;

// From Image 2
class ForceDisplacement {
protected:
double displacement, force;
public:
ForceDisplacement(double d, double f) {
displacement = d;
force = f;
}
double force_to_work() {
double W = force * displacement;
return W;
}
};

// From Image 2
class power_time {
protected:
double power;
int time;
public:
power_time(double pow, int t) {
power = pow;
time = t;
}
double power_to_work() {
double w = power * time;
return w;
}
};

// From Image 2
class angle_force {
protected:
double force, displacement, angle;
public:
angle_force(double f, double d, double a) {
force = f;
displacement = d;
angle = a;
```

```
}
double angle_to_work() {
// Note: cos expects radians in C++
double w = force * displacement * cos(angle);
return w;
}
};

// From Image 1
class work_energy {
protected:
int mess; // Typo in original code, likely meant 'mass'
int velocity;
public:
work_energy(int m, int v) {
mess = m;
velocity = v;
}
double energy_to_work() {
double w = 0.5 * (mess * (velocity * velocity));
return w;
}
};

// From Image 1
class work_gravity {
protected:
int mess; // Typo in original code
double gravity;
double height;
public:
work_gravity(int m, double g, double h) {
mess = m;
gravity = g;
height = h;
}
double gravity_to_work() {
double w = (mess * gravity * height);
return w;
}
};

// From Image 1
class elastic_force {
protected:
double constant;
double displacement;
public:
elastic_force(double k, double d) {
constant = k;
displacement = d;
```

```
}
double elasticForce_to_work() {
double w = 0.5 * (constant * (displacement * displacement));
return w;
}
};

// From Image 5
class thermodynamics {
protected:
double pressure, velocity1, velocity2;
public:
thermodynamics(double p, double v1, double v2) {
pressure = p;
velocity1 = v1;
velocity2 = v2;
}
double thermodynamics_work() {
double w = -pressure * (velocity1 - velocity2);
return w;
}
};

// From Image 5
class rotational_work {
protected:
double torque, angle;
public:
rotational_work(double tau, double a) {
torque = tau;
angle = a;
}
double rotational_to_work() {
double w = torque * angle;
return w;
}
};

// From Image 5 - The Main Derived Class
class work : public ForceDisplacement, public power_time, public angle_force,
public work_energy, public work_gravity, public elastic_force,
public thermodynamics, public rotational_work {
public:
work(double d, double f, double pow, int t, int m, double a, double g, int v,
double h, double k, double p, double v1, double v2, double tau)
: ForceDisplacement(d, f),
power_time(pow, t),
angle_force(f, d, a),
work_energy(m, v),
work_gravity(m, g, h),
elastic_force(k, d),
```

```
thermodynamics(p, v1, v2),
rotational_work(tau, a) // Reusing 'a' for rotational angle based on context
{
}

void measurement() {
double A = force_to_work();
double B = power_to_work();
double C = angle_to_work();
double D = energy_to_work();
double E = gravity_to_work();
double F = elasticForce_to_work();
double G = thermodynamics_work();
double H = rotational_to_work();

cout << "force_to_work() = " << A << endl;
cout << "power_to_work() = " << B << endl;
cout << "angle_to_work() = " << C << endl;
cout << "energy_to_work() = " << D << endl;
cout << "gravity_to_work() = " << E << endl;
cout << "elastic_force() = " << F << endl;
cout << "thermodynamical_work() = " << G << endl;
cout << "rotational_work() = " << H << endl;

// Comparison Logic from Image 4
if (A > B && A > C && A > D && A > E && A > F && A > G && A > H) {
cout << "A is greater than all" << endl;
} else if (B > A && B > C && B > D && B > E && B > F && B > G && B > H) {
cout << "B is greater than all" << endl;
} else if (C > A && C > B && C > D && C > E && C > F && C > G && C > H) {
cout << "C is greater than all" << endl;
} else if (D > A && D > B && D > C && D > E && D > F && D > G && D > H) {
cout << "D is greater than all" << endl;
} else if (E > A && E > B && E > C && E > D && E > F && E > G && E > H) {
cout << "E is greater than all" << endl;
} else if (F > A && F > B && F > C && F > D && F > E && F > G && F > H) {
cout << "F is greater than all" << endl;
} else if (G > A && G > B && G > C && G > D && G > E && G > F && G > H) {
cout << "G is greater than all" << endl;
} else if (H > A && H > B && H > C && H > D && H > E && H > F && H > G) {
cout << "H is greater than all" << endl;
} else {
cout << "No one is greater" << endl; // Handles ties
}
}
};

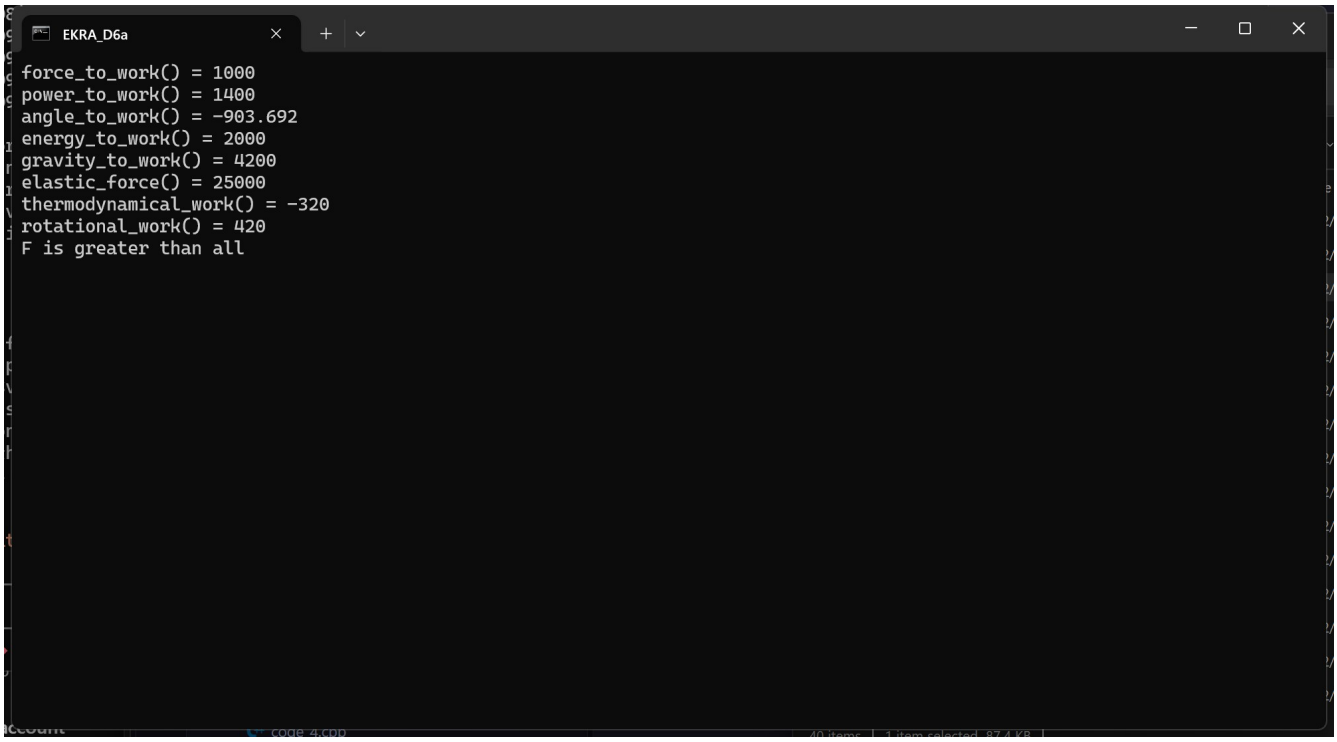
int main() {
// Arguments: d, f, pow, t, m, a(angle), g, v, h, k, p, v1, v2, tau
work obj(50, 20, 35, 40, 10, 35, 12, 20, 35, 20, 40, 50, 42, 12);
```

```
obj.measurement();

return 0;
}
```

Input:
(no input)

Output:



Program 2

Problem Statement
Demonstrate the diamond problem and its solution with virtual inheritance: base A is inherited virtually by B and C, and D inherits from both. Show that D holds a single shared copy of A and can display x, y and their sum.

Source Code:

```
#include <iostream>
using namespace std;

// Reconstructed Base Class A
class A {
protected:
int x;
public:
A() : x(0) {}
A(int a) {
x = a;
}
};

// Reconstructed Class B (Virtual Inheritance usually applies to both paths)
class B : virtual public A {
public:
B(int a) : A(a) {}
};

// From Image 3
class C : virtual public A {
protected:
int y;
public:
C(int b) {
y = b;
}
};

// From Image 3
class D : public B, public C {
public:
D(int a, int b) : B(a), C(b), A(a) { // A(a) is needed because A is virtual
}

void display() {
// This function name appears in main() but definition is missing in image.
// Assuming it displays x and y.
cout << "x: " << x << ", y: " << y << endl;
}

void add() {
cout << "Summation of two numbers: " << x + y << endl;
}
```

```
};

int main() {
D obj(2, 4);
obj.display();
obj.add();
return 0;
}
```

Input:
(no input)

Output:

